In [1]:

```
import pandas as pd
import json
with open('titanic.json') as f:
    df = pd.DataFrame(json.load(f))
df_clean = df.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])
df_clean['Age'] = df_clean['Age'].fillna(df_clean['Age'].median())
df_clean['Embarked'] = df_clean['Embarked'].fillna('S')
print(df_clean.head())
```

```
   Survived  Pclass     Sex   Age  SibSp  Parch     Fare Embarked
0         0       3    male  22.0      1      0   7.2500        S
1         1       1  female  38.0      1      0  71.2833        C
2         1       3  female  26.0      0      0   7.9250        S
3         1       1  female  35.0      1      0  53.1000        S
4         0       3    male  35.0      0      0   8.0500        S
```

In [2]:

```
import pandas as pd
df = pd.read_csv('Messy_Employee_dataset.csv')
df_clean = df.drop(columns=['Employee_ID', 'Email', 'Phone'])
df_clean['Age'] = df_clean['Age'].fillna(df_clean['Age'].median())
df_clean['Salary'] = pd.to_numeric(df_clean['Salary'], errors='coerce')
df_clean['Salary'] = df_clean['Salary'].fillna(df_clean['Salary'].median())
df_clean['Remote_Work'] = df_clean['Remote_Work'].astype(bool)
print(df_clean.head())
```

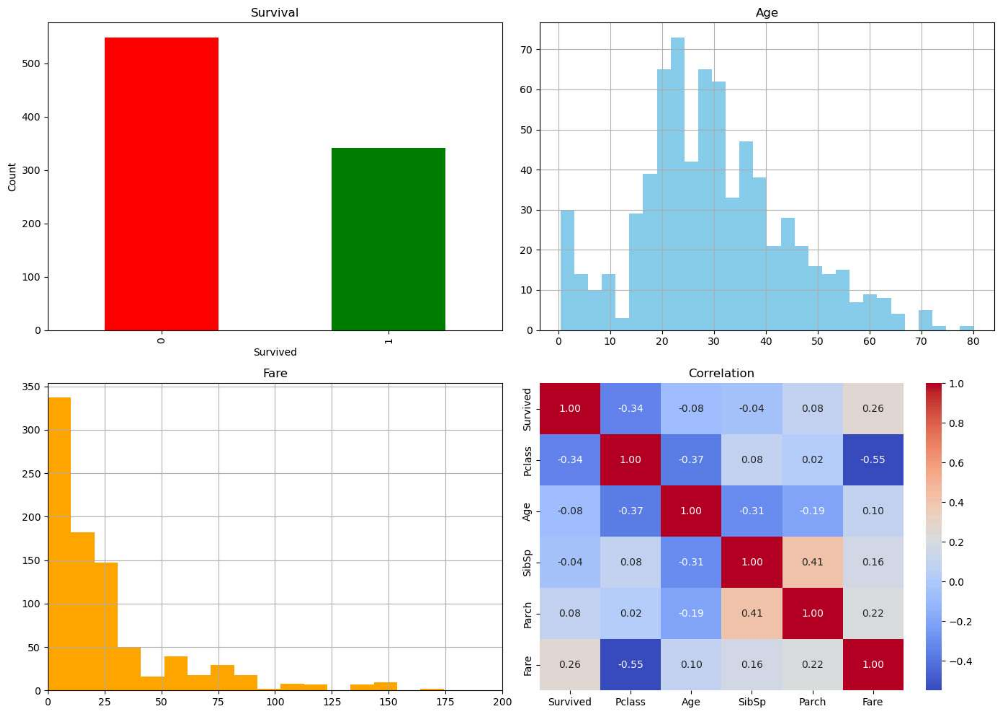
```
  First_Name Last_Name   Age   Department_Region    Status
Join_Date  \
0        Bob     Davis  25.0   DevOps-California    Active
4/2/2021
1        Bob     Brown  30.0       Finance-Texas    Active
7/10/2020
2      Alice     Jones  30.0        Admin-Nevada   Pending
12/7/2023
3        Eva     Davis  25.0        Admin-Nevada  Inactive
11/27/2021
4      Frank  Williams  25.0  Cloud Tech-Florida    Active
1/5/2022

      Salary Performance_Score  Remote_Work
0   59767.65           Average         True
1   65304.66         Excellent         True
2   88145.90              Good         True
3   69450.99              Good         True
4  109324.61              Poor        False
```

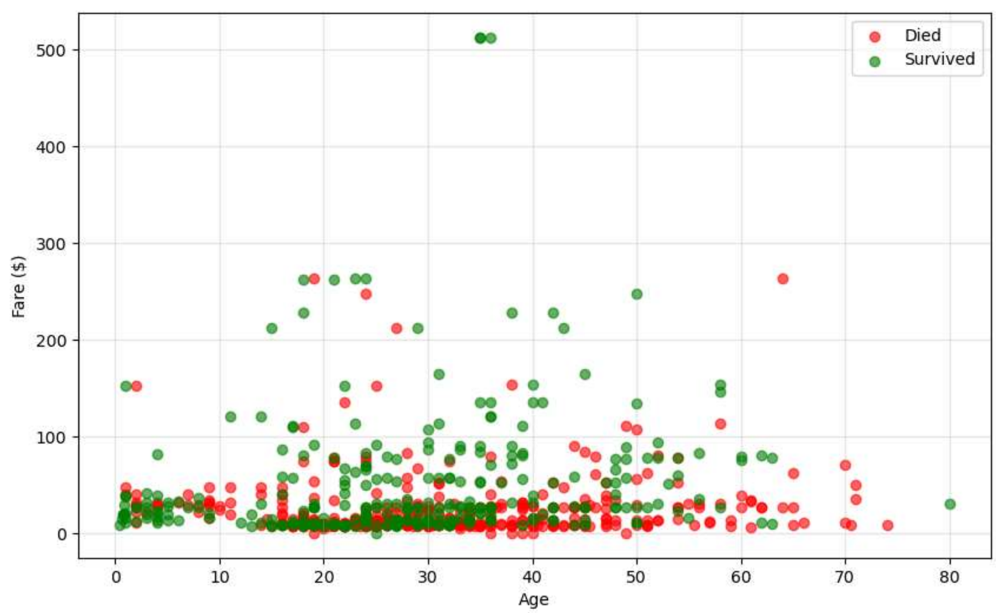
In [3]:

```
import pandas as pd

df = pd.read_csv('Messy_Employee_dataset.csv')
df = df.drop(columns=['Employee_ID', 'Email', 'Phone'])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = pd.to_numeric(df['Salary'],
errors='coerce').fillna(df['Salary'].median())
df['Remote_Work'] = df['Remote_Work'].astype(bool)
df = df.drop_duplicates()
df[['Department', 'Region']] = df['Department_Region'].str.split('-',
expand=True)
df = df.drop(columns=['Department_Region'])
avg_salary = df.groupby('Department')['Salary'].mean().reset_index()
avg_salary.columns = ['Department', 'Average_Salary']
print(avg_salary)
location = pd.DataFrame({
    'Department': ['DevOps', 'Finance', 'Admin', 'Cloud Tech', 'Sales',
'HR'],
    'Office_City': ['San Francisco', 'New York', 'Austin', 'Miami',
'Chicago', 'Seattle']
})
merged = df.merge(location, on='Department', how='left')
print(merged.head())
```

```
   Department  Average_Salary
0       Admin    85202.644096
1  Cloud Tech    84944.434658
2      DevOps    86003.638624
3     Finance    82279.519353
4          HR    85475.379474
5       Sales    86873.948258
  First_Name Last_Name   Age    Status   Join_Date     Salary  \
0        Bob     Davis  25.0    Active    4/2/2021   59767.65
1        Bob     Brown  30.0    Active   7/10/2020   65304.66
2      Alice     Jones  30.0   Pending   12/7/2023   88145.90
3        Eva     Davis  25.0  Inactive  11/27/2021   69450.99
4      Frank  Williams  25.0    Active    1/5/2022  109324.61

  Performance_Score  Remote_Work  Department      Region
Office_City
0           Average         True      DevOps  California  San
Francisco
1         Excellent         True     Finance       Texas       New
York
2              Good         True       Admin      Nevada
Austin
3              Good         True       Admin      Nevada
Austin
4              Poor        False  Cloud Tech     Florida
Miami
```

In [4]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json

df = pd.DataFrame(json.load(open('titanic.json')))

plt.figure(figsize=(14,10))
plt.subplot(2,2,1)
df['Survived'].value_counts().plot(kind='bar', color=['red','green'])
plt.title('Survival');plt.ylabel('Count')

plt.subplot(2,2,2)
df['Age'].dropna().hist(bins=30,color='skyblue')
plt.title('Age')

plt.subplot(2,2,3)
df['Fare'].hist(bins=50,color='orange');plt.xlim(0,200)
plt.title('Fare')

plt.subplot(2,2,4)
sns.heatmap(df[['Survived','Pclass','Age','SibSp','Parch','Fare']].corr(),
            annot=True,cmap='coolwarm',fmt='.2f')
plt.title('Correlation')
plt.tight_layout()
plt.show()

plt.figure(figsize=(10,6))
for s,c in [(0,'red'),(1,'green')]:
    d=df[df['Survived']==s].dropna(subset=['Age','Fare'])
    plt.scatter(d['Age'],d['Fare'],c=c,label=['Died','Survived']
[s],alpha=0.6)
plt.xlabel('Age');plt.ylabel('Fare
($)');plt.legend();plt.grid(True,alpha=0.3)
plt.show()
```





In [ ]:

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.